iscte
SINTRA
TECNOLOGIAS DIGITAIS
ECONOMIA E SOCIEDADE

Aplicação Front-end React

PROGRAMAÇÃO PARA A INTERNET

Beatriz Correia , 120105
Madalena Faias, 120111
Marta Feital, 120103
Matemática Aplicada à Transformação Digital, 3ºano
Ano letivo: 2024/2025

Conteúdo

Definição de Funcionalidades 2

Funcionalidades Implementadas 2

Exemplos de Código Implementado 3

Exemplo 1: Componente de Slider de House Cards na HomePage 3

Exemplo 2: Fetch dos Imóveis que estão na database 5

Exemplo 3: Forms do Sell 7

Explicação dos Routes Implementados 9

Overview dos Routes 9

Detalhes dos Routes 10

Definição de Funcionalidades

Funcionalidades Implementadas

1. **Sistema de Compra e Venda de Imóveis**
    - Permite aos utilizadores visualizar os apartamentos e casas disponíveis para compra, e permite ainda que os mesmo vendam as suas casas.
    - Implementado nas páginas "Buy" e "Sell", que consultam o ficheiro db.json para obter e registar imóveis.
2. **Favoritos**
    - Os utilizadores podem adicionar os imóveis aos favoritos e posteriormente visualizá-los na página "Favourites".
3. **Slider de imóveis com reviews superiores ou iguais a 4**
    - Interface dinâmica com cards dos imóveis destacados.
    - Implementado no ficheiro Slider.

Exemplos de Código Implementado

Exemplo 1: Componente de Slider de House Cards na HomePage

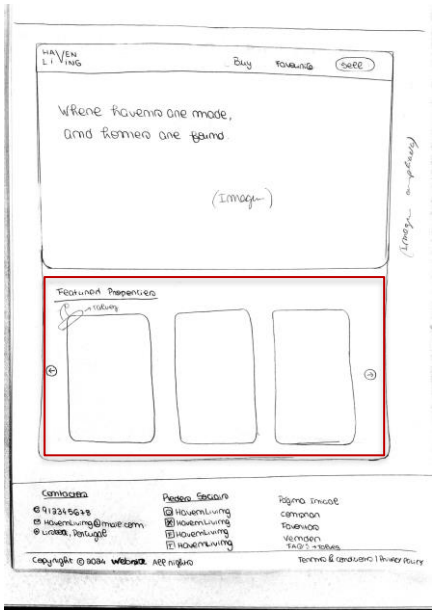


Figura 1-Mockup Home Page

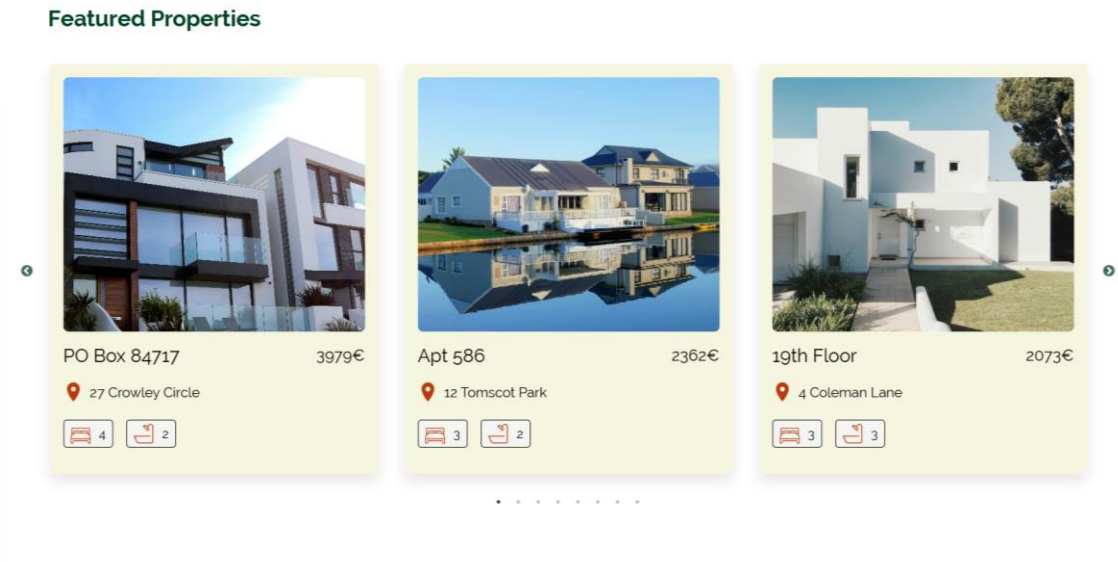


Figura 2- Slider Home Page Final

```
import React, { useEffect, useState } from "react";
```

```
import { Banner } from "../components/Banner";
import { HouseCard_HP } from "../components/HouseCard_HP";
import { Slider_ } from "../components/Slider";
import "./HomePage.css";

const getHouses = async () => {
    try {
        const response = await
fetch("http://localhost:3000/houses?review_gte=4");
      // Filtra casas com ranking >= 4
        const data = await response.json();
        setHouses(data);
    } catch (error) {
      console.error("Error fetching houses:", error);
    }
  };

<section className="house_container_hp">
        <h2 className="Featured Properties">Featured Properties</h2>
        <Slider_>
          {houses.map((house) => (
            <div key={house.id} className="slider_card">
              <HouseCard_HP {...house} context="home_card"
                onCardClick={() => navigate(`/apartment/${house.id}`)}
              />
            </div>
          ))}
        </Slider_>
      </section>
```

Explicação:

Este código define o componente da `HomePage.js`, que procura e exibe casas com avaliação mínima de 4 num slider (`Slider_`). Este importa `React`, `useEffect`, `useState` e componentes personalizados como `Banner` e `HouseCard_HP`. A função `getHouses` faz

uma requisição `fetch` ao servidor para obter os dados das casas, que são armazenados no estado `houses`. Esses dados são então mapeados para exibir cada casa dentro do `Slider_`, garantindo um identificador único para cada uma. Assim, a página inicial carrega dinamicamente as casas exibindo-as num slider interativo.

Exemplo 2: Fetch dos Imóveis que estão na database

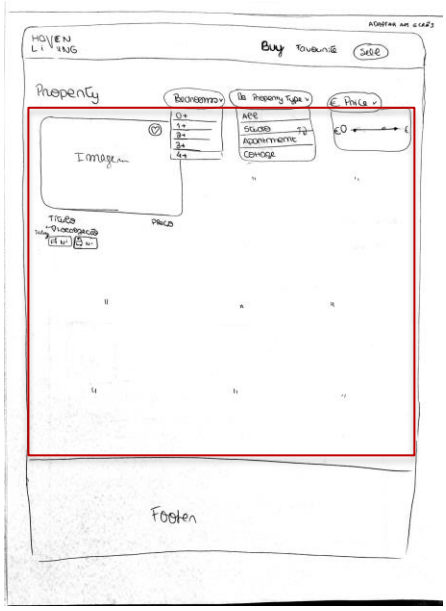


Figura 3-Mockup Buy Page

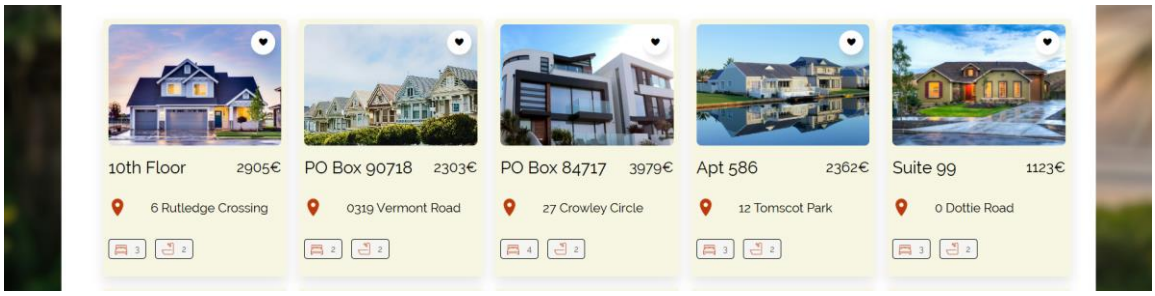


Figura 4- House Cards Buy Page Final

```
export function HouseCard_B({ onCardClick, toggleFavorite, isFavorite,
id, title, price, address, beds, baths }) {
```

```
    return (
        <div className="buy_card_b" onClick={onCardClick}>
            <div
                className="house_img"
                style={{ backgroundImage: `url(../imagens/${id}.jpg)` }}
            >
        <div className="house_content">
            <div className="house_title">
                <h3>{title}</h3>
                <h4>{price}€</h4>
            </div>
            <div className="house_location">
                <img      src="/icons/location.png"      alt="Location"
className="icon" />
                <p>{address}</p>
            </div>
            <div className="house_description">
                <div className="item">
                    <img         src="/icons/bed.png"         alt="beds"
className="icon" />
                    <span className="item_text">{beds}</span>
                </div>
                <div className="item">
                    <img        src="/icons/bath.png"        alt="baths"
className="icon" />
                    <span className="item_text">{baths}</span>
                </div>
            </div>
        </div>
```

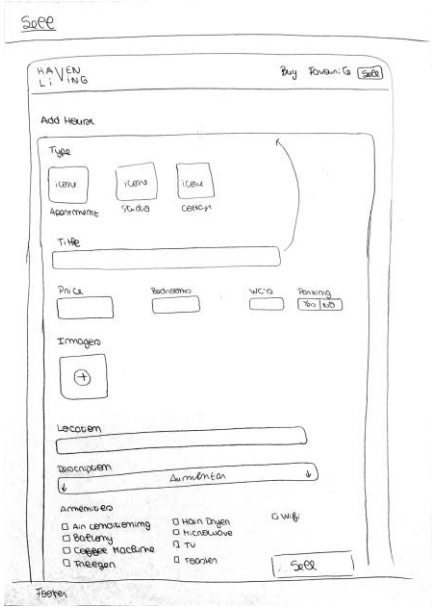
Explicação:

Este código define o componente `HouseCard_B`, que representa um cartão de uma casa disponível para compra. O componente recebe várias propriedades, incluindo `onCardClick` (função chamada ao clicar no cartão), `toggleFavorite` (função para marcar como favorito), `isFavorite` (estado do favorito), id, title, price, address, beds e baths.

O cartão é estruturado como uma div com a classe `buy_card_b`, indicando que é clicável. A imagem da casa é definida dinamicamente através da propriedade `backgroundImage`, usando o id da casa para buscar a imagem correspondente na pasta `../imagens/`. Dentro do cartão, há um bloco `house_content` que contém o título e o preço da casa, seguidos de uma secção `house_location`, onde um ícone de localização e o endereço são exibidos.

A parte final do cartão contém a secção `house_description`, onde são apresentados os ícones e números referentes ao número de quartos (beds) e casas de banho (baths).

Exemplo 3: Forms do Sell



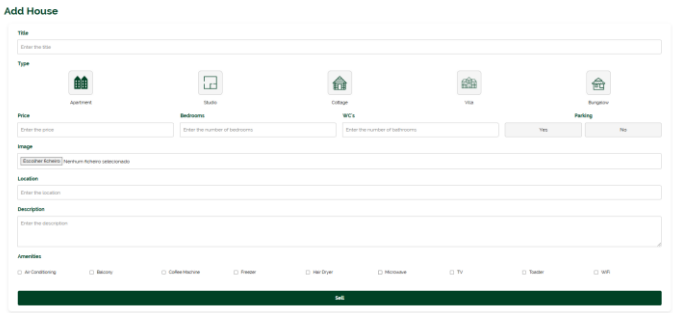


Figura 5- Sell Page

Figura 6- Sell Page Final

```
<div className="type-option-container">

    {[

        { type: "Apartment", icon: "/icons/apartment.png" },

        { type: "Studio", icon: "/icons/studio.png" },
```

```
        { type: "Cottage", icon: "/icons/cottage.png" },
        { type: "Villa", icon: "/icons/villa.png" },
        { type: "Bungalow", icon: "/icons/bungalow.png" },
    ].map(({ type, icon }) => (
        <div className="type-option" key={type}>
            <input
                type="radio"
                id={type}
                name="type"
                value={type}
                onChange={handleInputChange}
                required
            />
            <label htmlFor={type}>
                <img
                    src={icon}
                    alt={type}
                    className="type-icon"
                />
            </label>
            <span>{type}</span>
        </div>
    ))}
</div>
```

Explicação:

Este código define uma parte do formulário utilizado na página `Sell`. Esta é composta por um div principal com a classe `type-option-container`, de forma a apresentar opções de seleção dos diferentes tipos de propriedades. Cada opção é gerada dinamicamente a partir de um array de objetos, no qual cada objeto possui dois atributos: `type` e `icon`.

O método `.map()` percorre o array e cria um div para cada tipo de propriedade. Dentro de cada div, existe um input do tipo `radio` que permite a seleção de uma única opção. Posteriormente, este input é identificado pelos atributos id, name e value, que estão associados ao estado do formulário. A ligação é realizada através do evento onChange, acionado pela função handleInputChange, que atualiza o estado formData sempre que uma seleção é feita. O atributo required garante que o utilizador selecione obrigatoriamente uma opção antes de submeter o formulário.

Além do *input*, cada div inclui um label associado, definido pelo atributo htmlFor, que liga o rótulo ao *input* correspondente. Este label contém um elemento img, utilizado para atribuir o ícone representativo de cada tipo de propriedade, e um elemento span,

Explicação dos Routes Implementados

Overview dos Routes

- **Route 1:** `/`- Home Page, onde vemos o banner e o slogan da House Haven e abaixo estão as Featured Houses.
- **Route 2:** `/buy` - Buy Page onde encontramos todas as propriedades listadas para venda e é possível filtrá-las por type, number of bedrooms e price range
- **Route 3:** /apartment/{id} - Apartment Page onde encontramos todas as informações da casa selecionada, incluindo o nome e contacto do agente responsável por essa propriedade

- **Route 4:** /favourites - Favourite Properties onde estão as casas selecionadas como favoritas através do botão de favoritos que existe nos house cards e tal como a página buy tem a possibilidade de filtrar as mesmas.
- **Route 5:** /sell - Página onde é possível preencher um forms para adicionar uma casa ao catálogo de propriedades que se encontram à venda na House Haven.

Detalhes dos Routes

Este projeto é composto por 5 routes:

1. **Route: /**
2. **Route:** /buy
3. **Route:** /apartment/{id}
4. **Route:** /favourites
5. **Route:** /sell

No ficheiro `HomePage.js`, é utilizado um único método HTTP. O método GET é usado na função `getHouses` para obter uma lista de propriedades destacadas com base no critério de avaliação. Os parâmetros de entrada incluem `review_gte`, que filtra as propriedades cuja avaliação seja igual ou superior a 4. O pedido é enviado para o endpoint `http://localhost:3000/houses?review_gte=4`, e os dados retornados são processados no formato JSON e armazenados no estado local `houses`. Estes dados são utilizados para atribuir dinamicamente os cartões de propriedades em destaque na página inicial.

No ficheiro Buy.js, são utilizados dois métodos HTTP. O método GET usado na função `getHouses` para obter a lista de propriedades com base nos filtros aplicados, como tipo, número de quartos e faixa de preço. Os parâmetros de entrada incluem `type` para filtrar o tipo de propriedade, `beds_gte` para definir o número mínimo de quartos, `price_gte` para o limite inferior do preço e `price_lte` para o limite superior. O método PATCH é usado na função `toggleFavorite` para atualizar o estado de favorito de uma propriedade específica. Os parâmetros de entrada incluem o `id` da propriedade e um corpo em formato JSON que define o novo valor de `isFavorite` como true ou false.

No ficheiro ApartmentPage.js, são utilizados os métodos GET e PATCH. O método GET é usado na função `fetchDetails` para buscar os detalhes de uma propriedade específica com base no seu `id`, bem como para obter informações do agente associado à propriedade, utilizando o `agentId`. O método PATCH é usado na função `toggleFavorite` para alternar o estado de favorito da propriedade, recebendo como parâmetro o `id` da propriedade e um corpo em formato JSON com o novo valor de `isFavorite`.

No ficheiro Favourites.js, também são utilizados os métodos GET e PATCH. O GET é usado na função `getHouses` para mostrar propriedades favoritas aplicandos filtros, semelhantes ao ficheiro Buy.js, com a adição do parâmetro `isFavorite=true` para garantir que apenas as propriedades favoritas sejam retornadas. O método PATCH, utilizado na função `toggleFavorite`, permite alternar o estado de favorito de uma propriedade específica, recebendo o `id` da propriedade como parâmetro e um corpo em formato JSON com o novo valor de `isFavorite`.

Por fim, no ficheiro Sell.js, são utilizados os métodos GET e POST. O método GET é usado na função `handleSubmit` para pesquisar todas as propriedades existentes, calculando o próximo ID único para uma nova propriedade. Enquanto, o método POST é utilizado para adicionar uma nova propriedade ao servidor, enviando no corpo da requisição informações como título, morada, preço, número de quartos, número de casas de banho, estado do estacionamento, descrição, tipo, lista de amenidades e nome da imagem.